

MATH 8090: ARMA Models: Prediction and Forecasting, Modeling Case Study

Whitney Huang, Clemson University

9/28-10/2/2026

Contents

NOAA wind data example	1
Load and plot the data	2
“Estimate” ϕ using sample ACF and center the data	3
One-step-ahead forecast	5
Fill in missing value example	6
Simulate an AR(-0.9)	6
Let’s remove some data to illustrate how to fill in missing values using forecasting algorithm	8
Fill in “missing” values	9
Prediction Errors from Best Linear Predictor	10
Ireland wind data case study	11
Prepare the wind-speed data	11
Why use a square-root transformation?	11
Does the transformation stabilize seasonal variability?	13
Identify the remaining temporal dependence	14
Jointly estimate seasonality and temporal dependence	15
Compare the candidate models	16
Diagnose the selected model	16
Forecast future wind speed	17
Plot the 10-Day Wind-Speed Forecast	19

NOAA wind data example

This example is taken from Don Percival’s time series course (UW Stat 519).

The one-step-ahead forecast of an AR(1) process is:

$$P_n X_{n+1} = \hat{\mu} + \hat{\phi}(X_n - \hat{\mu}),$$

where $\hat{\phi}$ is our estimate of ϕ , and $\hat{\mu}$ is an estimate of μ .

Load and plot the data

```
# Read the wind speed time series from the online data file
ws <- scan("http://faculty.washington.edu/dbp/s519/Data/wind-speed.txt")

# Determine the number of observations
n <- length(ws)

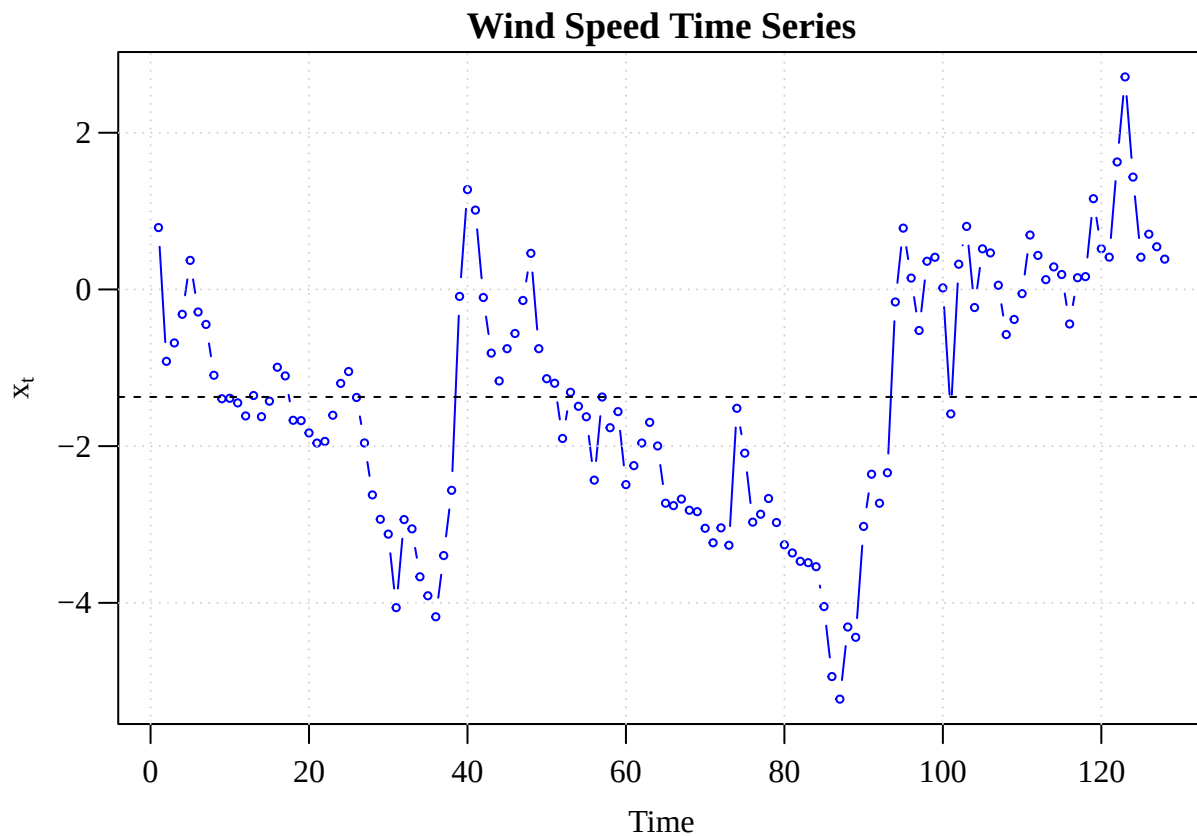
# Set graphical parameters for the time series plot
par(las = 1, mgp = c(2, 0.7, 0), mar = c(3.6, 3.6, 1.4, 0.6),
    family = "serif")

# Plot the wind speed observations over time
plot(ws, col = "blue", xlab = "Time", type = "b",
     ylab = expression(x[t]), main = "Wind Speed Time Series", cex = 0.5)

# Add grid lines to the plot
grid()

# Calculate the sample mean of the wind speed series
xbar_ws <- mean(ws)

# Add the sample mean as a horizontal dashed line
abline(h = xbar_ws, lty = 2)
```



“Estimate” ϕ using sample ACF and center the data

```
# --- Estimate the AR(1) coefficient using the lag-1 sample ACF -----
# For an AR(1) process, rho(1) = phi, so the lag-1 sample ACF provides
# a simple moment-based estimate of the AR coefficient
acf.ws <- acf(ws, lag.max = 40, plot = FALSE)$acf # includes lag 0
phi.ws <- acf.ws[2]                               # lag-1 sample ACF

# --- Bartlett variance factor for the sample ACF under an AR(1) model -----
# Compute w_hh(phi), where the approximate sampling variance is
# Var(rho_hat(h)) w_hh(phi) / n
gen.whh.ar <- function(h, phi){
  p2 <- phi^2
  p2h <- p2^h
  -2 * h * p2h + (1 - p2h) * (1 + p2) / (1 - p2)
}

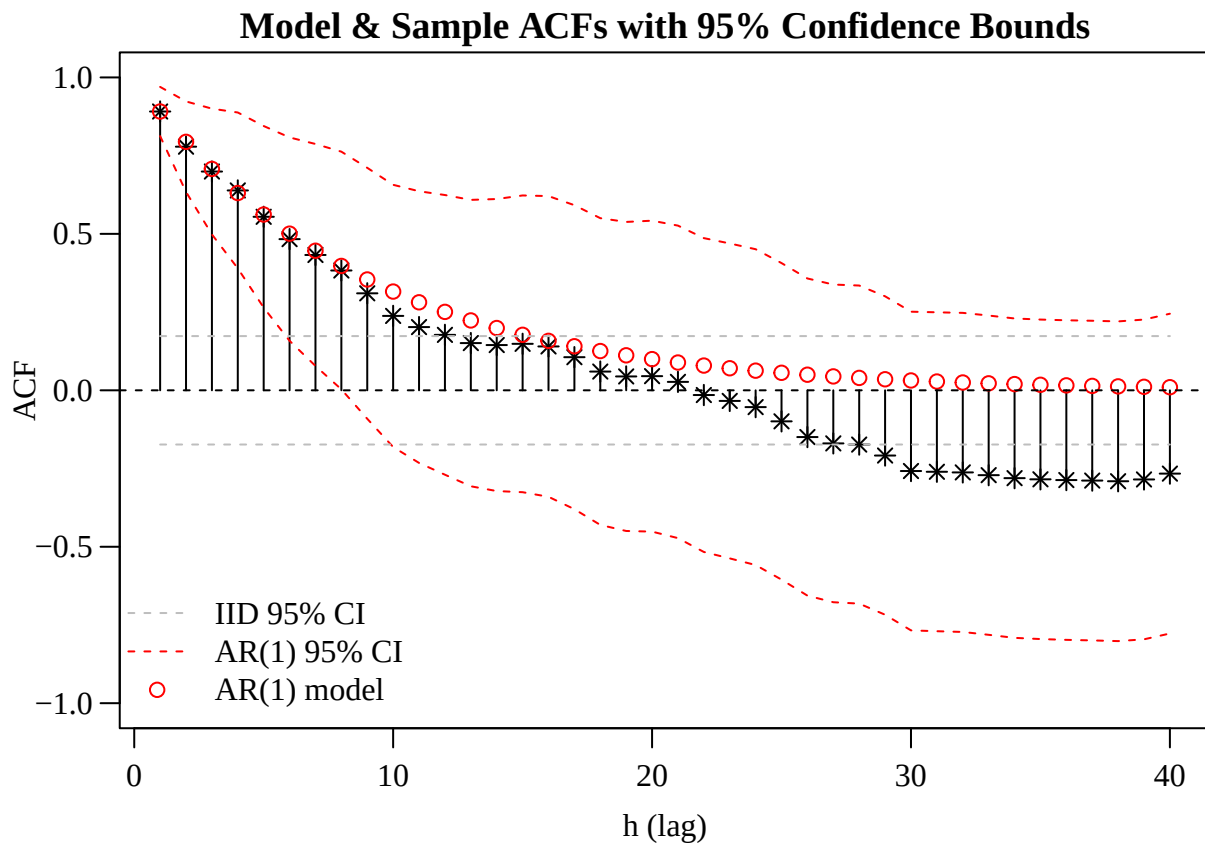
# --- Compare the sample ACF with the fitted AR(1) ACF -----
# Plot the sample ACF together with:
# 1. the theoretical ACF implied by the fitted AR(1) model,
# 2. Bartlett-based 95% confidence bounds under the AR(1) model, and
# 3. the usual IID 95% confidence bounds for comparison
plot.ACFbartlettAR <- function(ts, n.lags = 40){
  stopifnot(is.numeric(ts), length(ts) > 5)
  n.ts <- length(ts)
  lags <- 1:n.lags
  # Compute the sample ACF for lags 1,...,n.lags (excluding lag 0)
  acf.full <- acf(ts, lag.max = n.lags, plot = FALSE)$acf
  acf.est <- acf.full[-1]
  # Estimate phi by the lag-1 sample ACF and obtain the corresponding
  # theoretical AR(1) ACF: rho(h) = phi^h
  phi.hat <- acf.est[1]
  acf.model <- phi.hat^lags
  # Plot the sample ACF using vertical stems and star-shaped points
  plot(lags, acf.est, type = "h",
       xlab = "h (lag)", ylab = "ACF",
       ylim = c(-1, 1),
       main = "Model & Sample ACFs with 95% Confidence Bounds",
       las = 1)
  points(lags, acf.est, pch = 8)
  # Add the theoretical ACF implied by the fitted AR(1) model
  points(lags, acf.model, col = "red")
  # Add Bartlett-based 95% confidence bounds under the AR(1) model
  CI.AR <- 1.96 * sqrt(sapply(lags, function(h)
    gen.whh.ar(h, phi.hat))) / sqrt(n.ts)
  lines(lags, acf.est + CI.AR, col = "red", lty = 2)
  lines(lags, acf.est - CI.AR, col = "red", lty = 2)
  # Add the usual IID 95% confidence bounds, ±1.96/sqrt(n), for comparison
  CI.IID <- rep(1.96 / sqrt(n.ts), n.lags)
  lines(lags, -CI.IID, col = "gray", lty = 2)
  lines(lags, CI.IID, col = "gray", lty = 2)
  # Add the zero-reference line and legend
  abline(h = 0, lty = "dashed")
```

```

legend("bottomleft",
      legend = c("IID 95% CI", "AR(1) 95% CI", "AR(1) model"),
      lty     = c(2, 2, NA),
      pch     = c(NA, NA, 1),
      col     = c("gray", "red", "red"),
      bty     = "n")
}

# --- Set plotting parameters and display the ACF comparison -----
par(mgp = c(2, 0.7, 0), mar = c(3.5, 3.5, 1.4, 0.6), family = "serif")
plot.ACFbartlettAR(ws)

```



```

# --- Alternatively, estimate the AR(1) model by maximum likelihood -----
# Fit an AR(1) model directly rather than estimating phi from the lag-1 ACF
phi_hat_mle <- arima(ws, order = c(1, 0, 0))
phi_hat_mle

```

```

##
## Call:
## arima(x = ws, order = c(1, 0, 0))
##
## Coefficients:
##          ar1  intercept
##         0.906   -1.1136
## s.e.  0.037    0.6035

```

```
##
## sigma^2 estimated as 0.4615: log likelihood = -132.99, aic = 271.99

# --- Center the series if a mean-zero representation is needed -----
# Subtract the sample mean so that the transformed series has sample mean zero
ws.centered <- ws - mean(ws)
```

One-step-ahead forecast

```
# -- One-step-ahead predictions and errors # --- One-step-ahead prediction using the fitted AR(1) model

# For t = 2,...,n, predict x_t using the previous observation x_{t-1}
# The mean-adjusted AR(1) predictor is:
# x_hat[t] = xbar + phi_hat * (x[t-1] - xbar)
ws.hat <- phi.ws * ws.centered[1:(n - 1)] + xbar_ws

# Compute one-step-ahead prediction errors:
# e_t = x_hat[t] - x[t]
zt.ws <- ws.hat - ws[2:n]

# --- Plot the observed series and one-step-ahead predictions -----
par(las = 1, mgp = c(2, 0.7, 0), mar = c(3.5, 3.5, 1.2, 0.6),
    family = "serif")

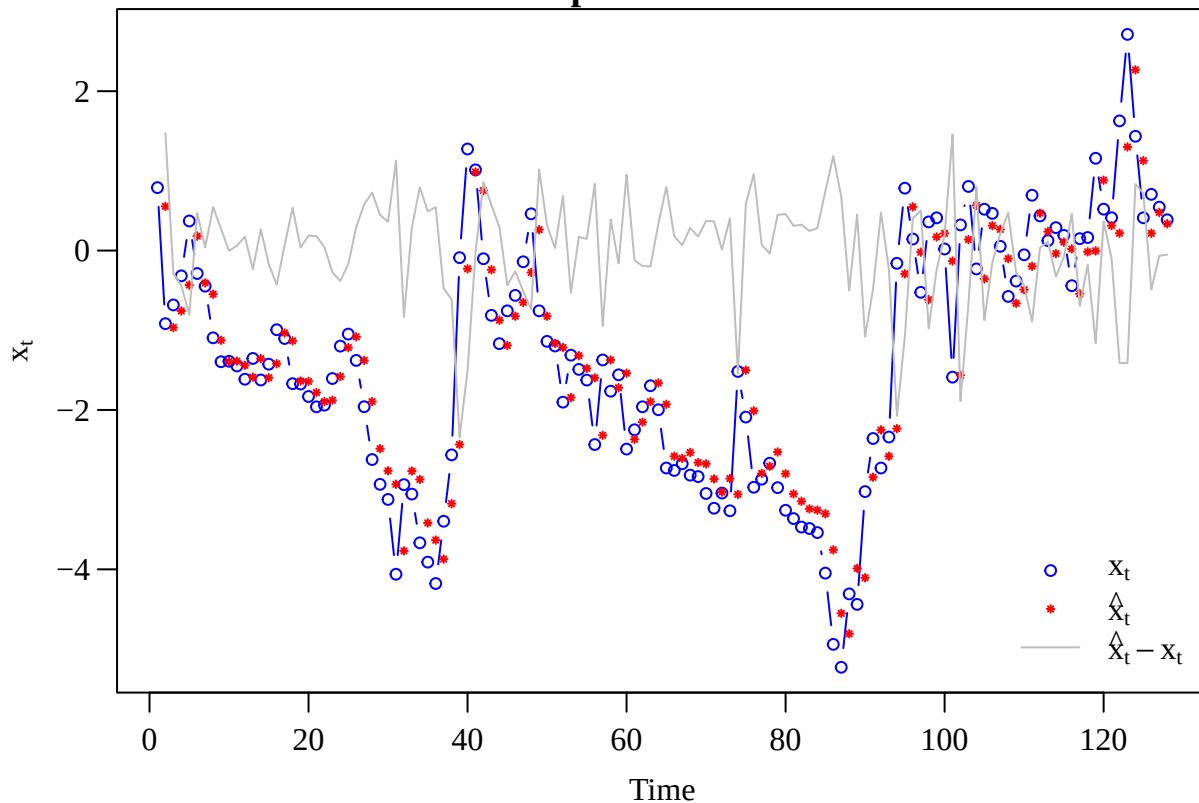
# Plot the observed wind speed time series
plot(ws, col = "blue", xlab = "Time", type = "b",
     ylab = expression(x[t]),
     main = "One-Step-Ahead Prediction", cex = 0.75)

# Add the one-step-ahead predictions for t = 2,...,n
points(2:n, ws.hat, pch = 8, col = "red", cex = 0.375)

# Add the corresponding one-step-ahead prediction errors
lines(2:n, zt.ws, col = "gray")

# Add a legend identifying the observations, predictions, and errors
legend("bottomright",
     legend = expression(x[t], hat(x)[t], hat(x)[t] - x[t]),
     col = c("blue", "red", "gray"),
     pch = c(1, 8, NA), lty = c(NA, NA, "solid"),
     pt.cex = c(0.75, 0.375, 1), inset = 0.01, bty = "n")
```

One-Step-Ahead Prediction



--- Compare prediction-error variance with the variance of the series -----

*# A useful AR(1) model should reduce uncertainty relative to predicting
without using the previous observation; compare the prediction-error
variance with the marginal variance of the original series*

```
cat("\nQuick AR(1) estimate (lag-1 ACF): phi_hat =", round(phi.ws, 4), "\n")
```

```
##
```

```
## Quick AR(1) estimate (lag-1 ACF): phi_hat = 0.8911
```

```
cat("Var(prediction errors) =", round(var(zt.ws), 4), "\n")
```

```
## Var(prediction errors) = 0.4629
```

```
cat("Var(series)          =", round(var(ws), 4), "\n")
```

```
## Var(series)          = 2.5025
```

Fill in missing value example

Simulate an AR(-0.9)

```

# Generate 100 observations from the AR(1) model:
#  $X_t = \phi * X_{t-1} + Z_t$ ,  $Z_t \sim N(0, 1)$ 
generate.AR1.ts <- function(phi = 0.0){
  ts <- rep(0, 100)
  # Initialize  $X_1$  from the stationary distribution
  # For innovation variance 1,  $\text{Var}(X_t) = 1 / (1 - \phi^2)$ 
  ts[1] <- rnorm(1) / sqrt(1 - phi^2)
  # Generate subsequent observations recursively from the AR(1) model
  for(i in 2:100) ts[i] <- phi * ts[i - 1] + rnorm(1)
  ts
}

# Set the random seed for reproducibility
set.seed(123)

# Simulate an AR(1) process with  $\phi = -0.9$ 
# A negative  $\phi$  produces an oscillating pattern between successive observations
ar1.ts <- generate.AR1.ts(-0.9)

# Plot the simulated AR(1) time series

# Load scales to add transparency to the connecting line
library(scales)

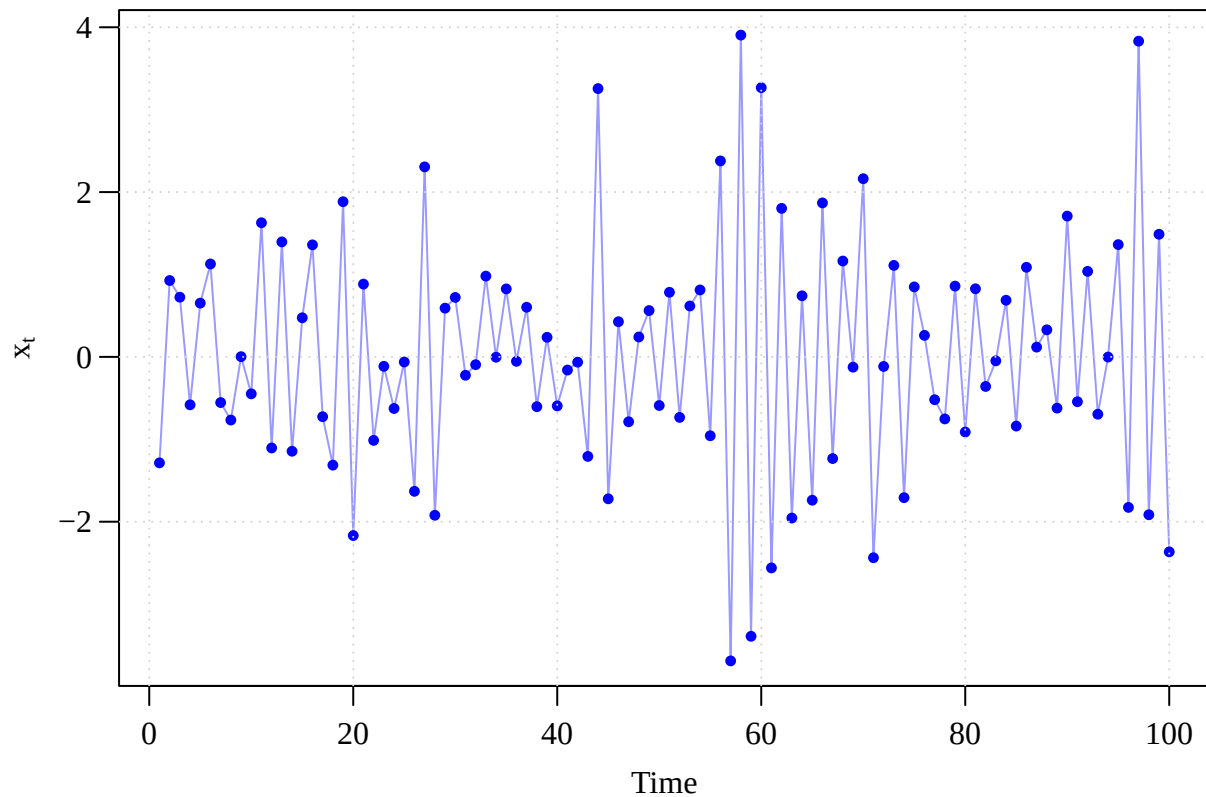
# Set graphical parameters
par(las = 1, mgp = c(2, 0.7, 0), mar = c(3.5, 3.5, 1.4, 0.6),
    family = "serif")

# Plot the time series with a semi-transparent line
plot(ar1.ts, col = alpha("blue", 0.4), xlab = "Time", type = "l",
     ylab = expression(x[t]), cex = 0.5)

# Add points to highlight individual observations
points(ar1.ts, pch = 16, cex = 0.75, col = "blue")

# Add grid lines for easier visual comparison across time
grid()

```



Let's remove some data to illustrate how to fill in missing values using forecasting algorithm

```
# Subsample the AR(1) series by retaining every other observation

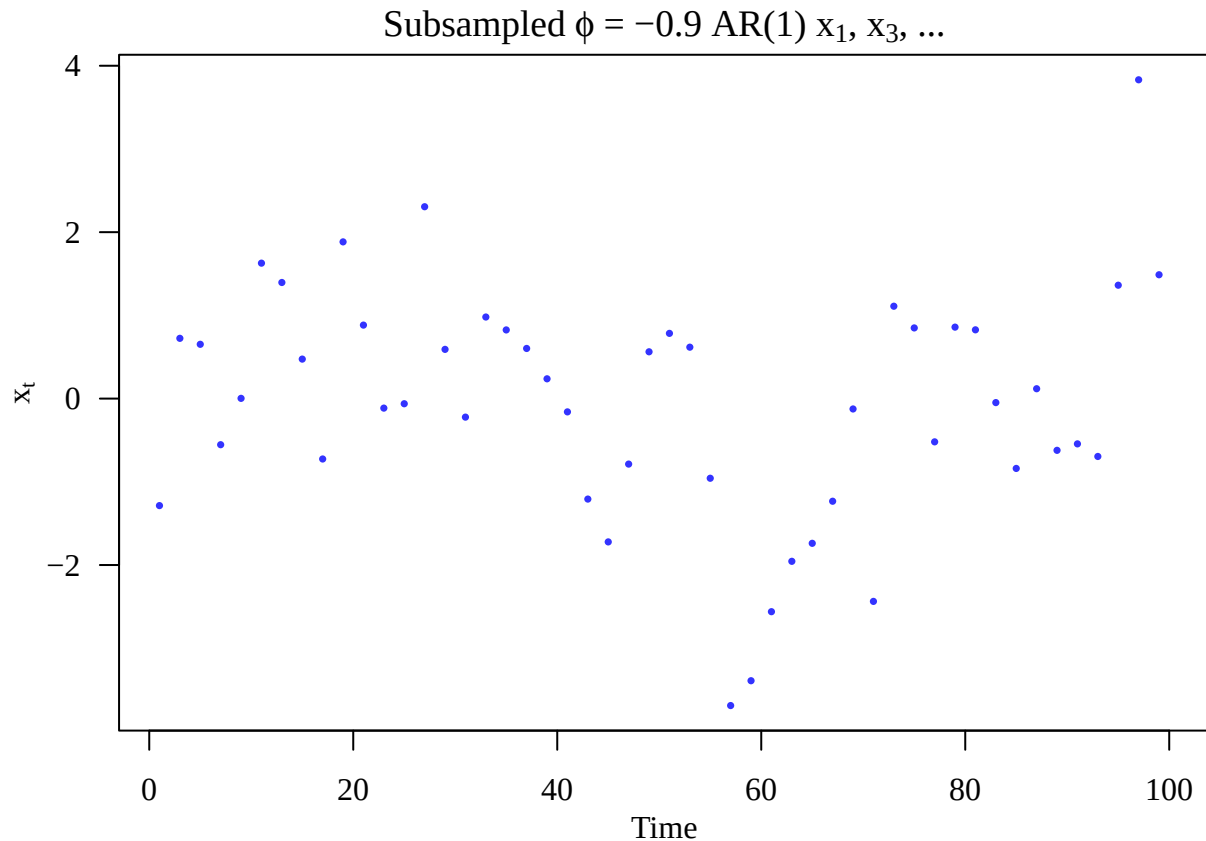
# Create a copy of the original AR(1) series
ar1.ts.subsampled <- ar1.ts

# Remove observations at even time points, leaving X_1, X_3, X_5, ...
# The retained observations are therefore separated by two time steps
ar1.ts.subsampled[seq(2, 100, 2)] <- NA

# Plot the subsampled AR(1) time series

# Set graphical parameters
par(las = 1, mgp = c(2, 1, 0), mar = c(3.5, 3.5, 1.4, 0.6),
    family = "serif")

# Plot the retained observations X_1, X_3, X_5, ...
# For an AR(1) process,  $\text{Corr}(X_t, X_{t-2}) = \phi^2$ ;
# thus subsampling changes the dependence between consecutive retained values
plot(ar1.ts.subsampled, xlab = "Time", type = "b",
     ylab = expression(x[t]),
     main = expression(paste("Subsampled ", phi, " = -0.9 AR(1) ",
                             x[1], ", ", x[3], ", ...")),
     cex = 0.5, col = alpha("blue", 0.8), pch = 16)
```



Fill in “missing” values

$$\hat{X}_2 = \phi(X_1 + X_3)/(1 + \phi^2)$$

$$\text{MSPE} = \frac{\sigma^2}{1 + \phi^2}$$

```
# Copy the original AR(1) series for prediction
ar1.ts.predicted <- ar1.ts

# Predict the missing even-indexed observations using their two neighbors
# Conditional expectation for an AR(1) process with phi = -0.9
ar1.ts.predicted[seq(2, 98, 2)] <-
  -0.9 * (ar1.ts[seq(1, 97, 2)] + ar1.ts[seq(3, 99, 2)]) / 1.81

# No two-sided prediction is available for the last observation
ar1.ts.predicted[100] <- NA

# Set plotting parameters
par(las = 1, mgp = c(2, 1, 0), mar = c(3.5, 3.5, 1.4, 0.6),
    family = "serif")

# Plot the series with predicted values filled in
plot(ar1.ts.predicted, col = alpha("blue", 0.4), xlab = "Time", type = "l",
     ylab = expression(x[t]), cex = 0.5)

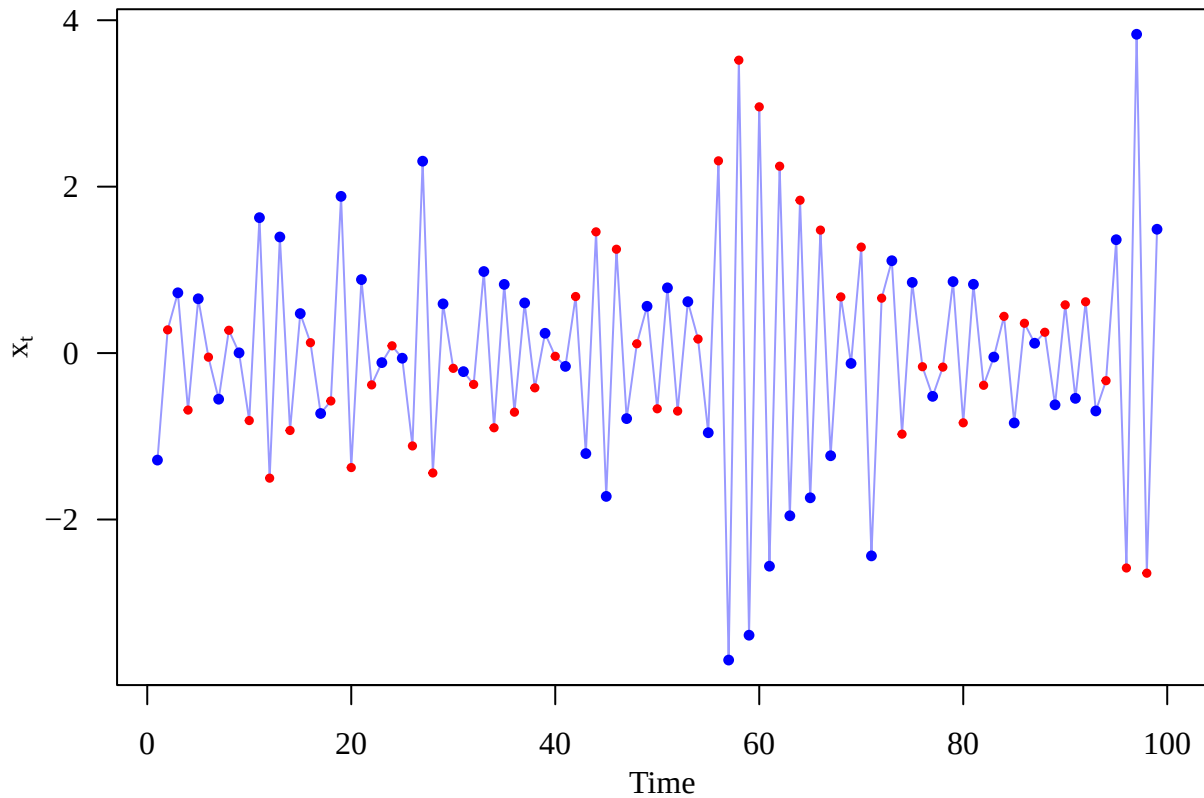
# Highlight predicted observations in red
```

```

xs <- seq(2, 98, 2)
points(xs, ar1.ts.predicted[xs], pch = 19, col = "red", cex = 0.5)

# Highlight observed observations in blue
xo <- seq(1, 99, 2)
points(xo, ar1.ts.subsampled[xo], col = "blue", cex = 0.75, pch = 16)

```



Prediction Errors from Best Linear Predictor

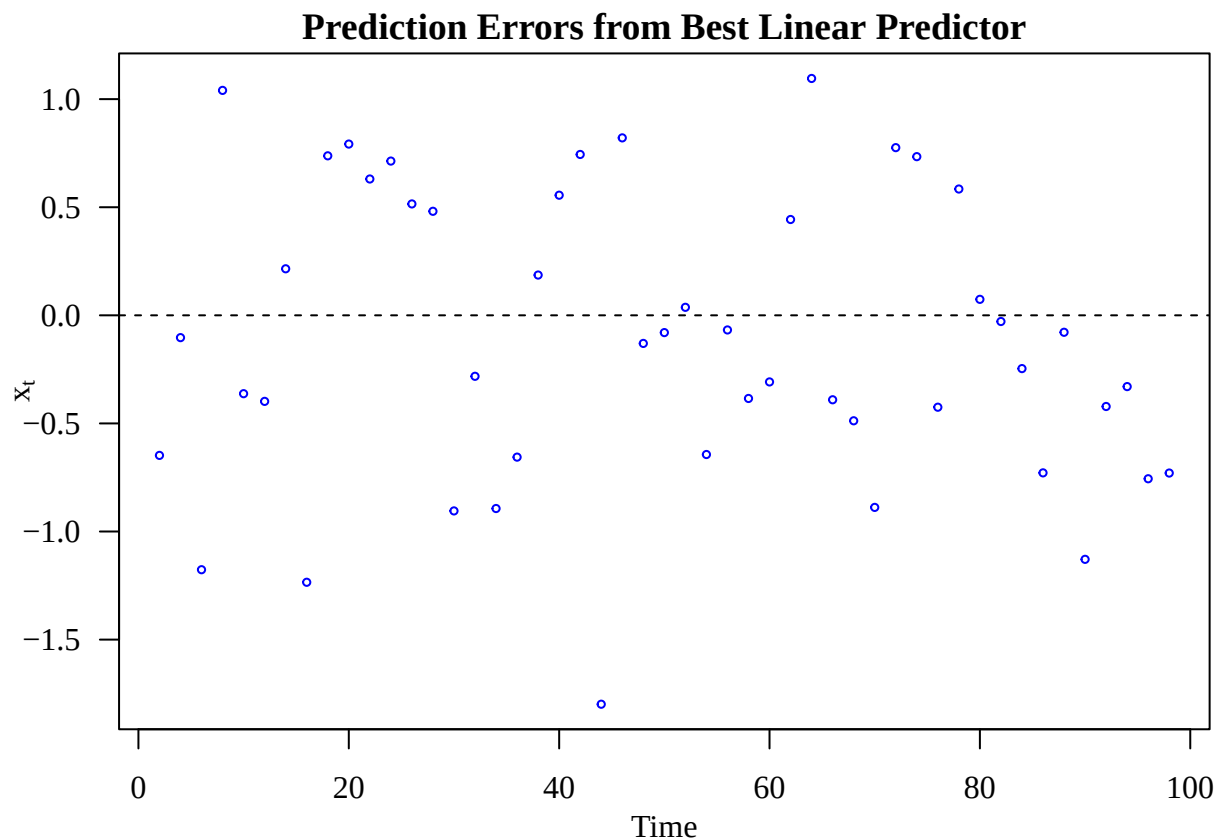
```

# Set plotting parameters
par(las = 1, mgp = c(2, 1, 0), mar = c(3.5, 3.5, 1.4, 0.6),
    family = "serif")

# Plot prediction errors at the missing even-indexed time points
plot(xs, (ar1.ts.predicted - ar1.ts)[xs], col = "blue",
     xlab = "Time", type = "p", ylab = expression(x[t]),
     main = "Prediction Errors from Best Linear Predictor", cex = 0.5)

# Add a reference line at zero prediction error
abline(h = 0, lty = 2)

```



Ireland wind data case study

Prepare the wind-speed data

We begin with daily wind-speed observations at Rosslare from 1965–1969. We construct the calendar date, retain the raw wind speed, and define its square-root transformation. The month indicator will later be used to examine whether the transformation stabilizes variability across the annual cycle.

```
library(gstat)
data(wind)

# Rosslare daily wind-speed observations, 1965--1969
dat <- wind[wind$year %in% 65:69, ]

date <- as.Date(sprintf("%04d-%02d-%02d", dat$year + 1900, dat$month, dat$day))

wind_speed <- dat$ROS
sqrt_wind <- sqrt(wind_speed)

# Month labels used for seasonal variance comparisons
month <- factor(format(date, "%b"), levels = month.abb)
```

Why use a square-root transformation?

Wind speed is nonnegative and may exhibit right skewness. We first compare the marginal distributions of the raw and square-root-transformed observations. Histograms assess symmetry and overall distributional

shape, while normal Q-Q plots provide a more direct comparison with a Gaussian distribution. The goal is not to require the marginal wind-speed observations to be exactly Gaussian. Rather, this provides an initial diagnostic for whether the square-root transformation produces a distribution more compatible with the Gaussian error model used later.

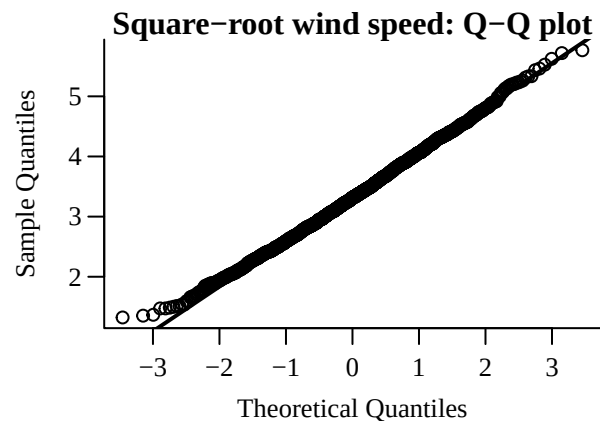
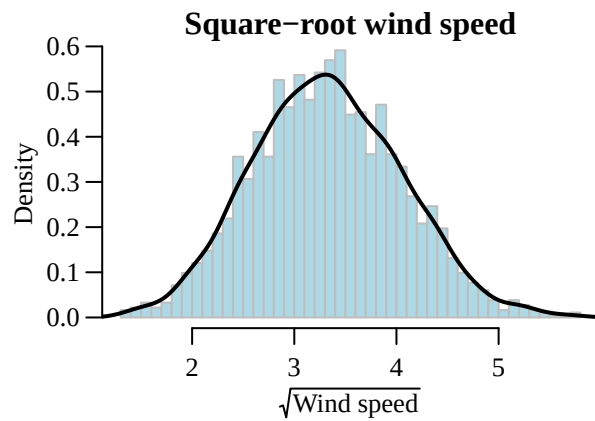
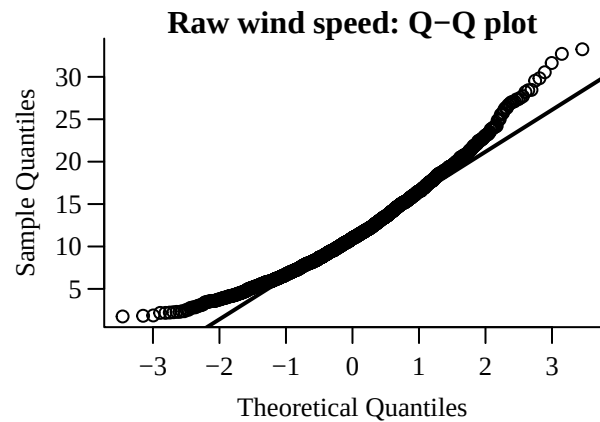
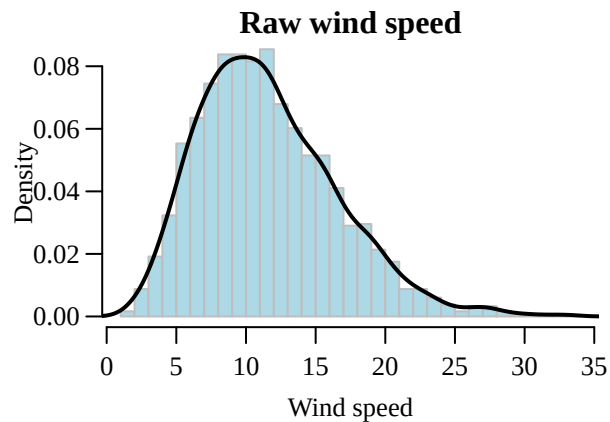
```
par(mfrow = c(2, 2), bty = "L", mar = c(3.5, 3.5, 1, 0.5),
    mgp = c(2, 0.7, 0), family = "serif", las = 1)

# Raw wind speed
hist(wind_speed, prob = T, breaks = 40, col = "lightblue", border = "gray",
     main = "Raw wind speed", xlab = "Wind speed")
lines(density(wind_speed), lwd = 2)

qqnorm(wind_speed, main = "Raw wind speed: Q-Q plot")
qqline(wind_speed, lwd = 2)

# Square-root-transformed wind speed
hist(sqrt_wind, prob = T, breaks = 40, col = "lightblue", border = "gray",
     main = "Square-root wind speed",
     xlab = expression(sqrt("Wind speed")))
lines(density(sqrt_wind), lwd = 2)

qqnorm(sqrt_wind, main = "Square-root wind speed: Q-Q plot")
qqline(sqrt_wind, lwd = 2)
```



```
par(mfrow = c(1, 1))
```

Does the transformation stabilize seasonal variability?

A second motivation for transformation is variance stabilization. Because wind speed has a strong annual cycle, we should not compare monthly variability before accounting for changes in the seasonal mean. We represent the annual seasonal mean using $K = 4$ Fourier harmonics,

$$s(t) = \beta_0 + \sum_{k=1}^4 \left[a_k \cos\left(\frac{2\pi kt}{365.25}\right) + b_k \sin\left(\frac{2\pi kt}{365.25}\right) \right].$$

The same seasonal structure is fitted on the raw and square-root scales so that the residual variability can be compared fairly.

```
# Time in days from the beginning of the series
origin <- date[1]
time_day <- as.numeric(date - origin)

# Construct Fourier regressors for annual seasonality
harmonic_matrix <- function(t, K, period = 365.25) {
  angle <- outer(2 * pi * t / period, seq_len(K))
  X <- cbind(cos(angle), sin(angle))

  colnames(X) <- c(paste0("cos", seq_len(K)),
                   paste0("sin", seq_len(K)))
  X
}

K <- 4
X <- harmonic_matrix(time_day, K)

# Fit the same seasonal mean structure on both scales
fit_raw <- lm(wind_speed ~ X)
fit_sqrt <- lm(sqrt_wind ~ X)

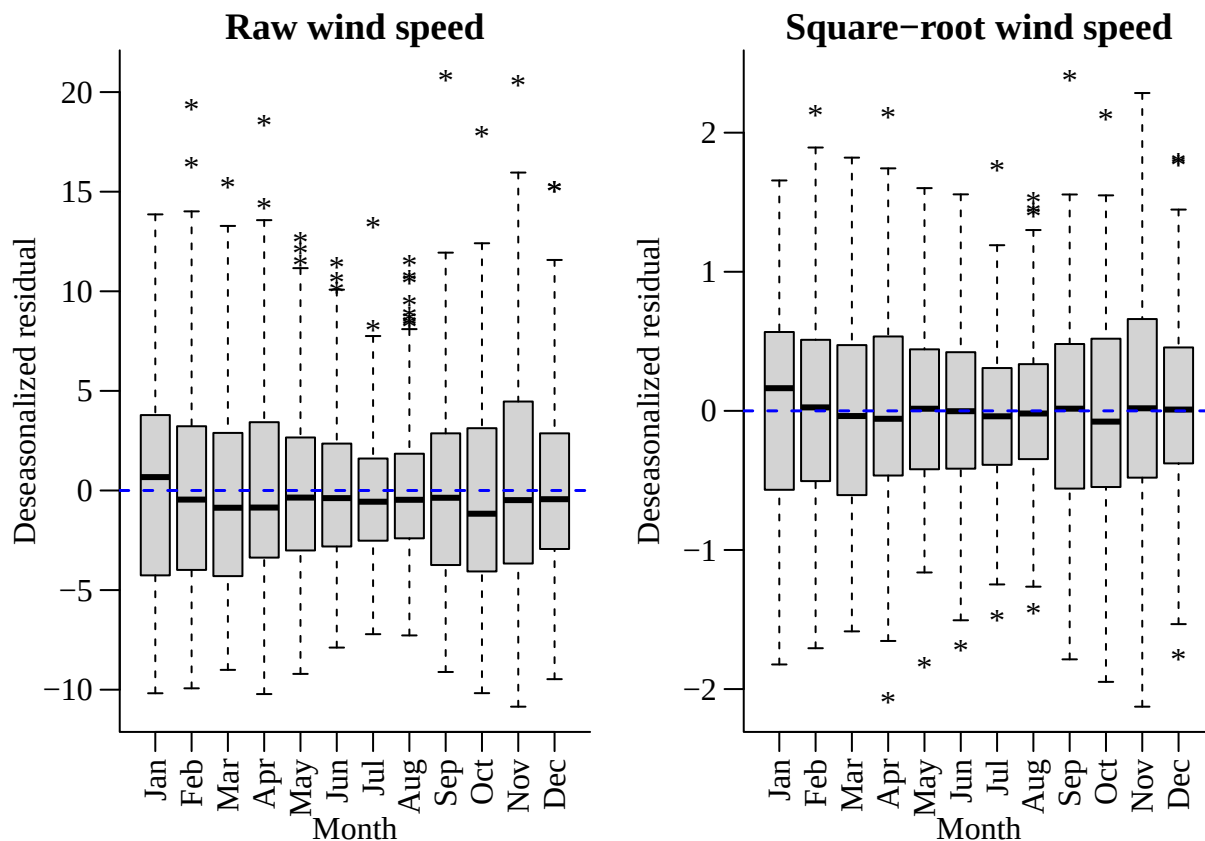
res_raw <- residuals(fit_raw)
res_sqrt <- residuals(fit_sqrt)
```

We now compare the deseasonalized residuals by month. If the residual spread varies substantially across months on the raw scale but becomes more homogeneous after transformation, this provides evidence that the square-root transformation stabilizes the variance.

```
par(mfrow = c(1, 2), bty = "L", mar = c(3.5, 3.5, 1.25, 0.5),
    mgp = c(2, 0.7, 0), las = 2, family = "serif")

boxplot(res_raw ~ month, main = "Raw wind speed",
        xlab = "Month", ylab = "Deseasonalized residual", outpch = "*")
abline(h = 0, lty = 2, lwd = 1.5, col = "blue")

boxplot(res_sqrt ~ month, main = "Square-root wind speed",
        xlab = "Month", ylab = "Deseasonalized residual", outpch = "*")
abline(h = 0, lty = 2, lwd = 1.5, col = "blue")
```



```
par(mfrow = c(1, 1))
```

If these diagnostics support the transformation, we proceed with the square-root wind speed as the response:

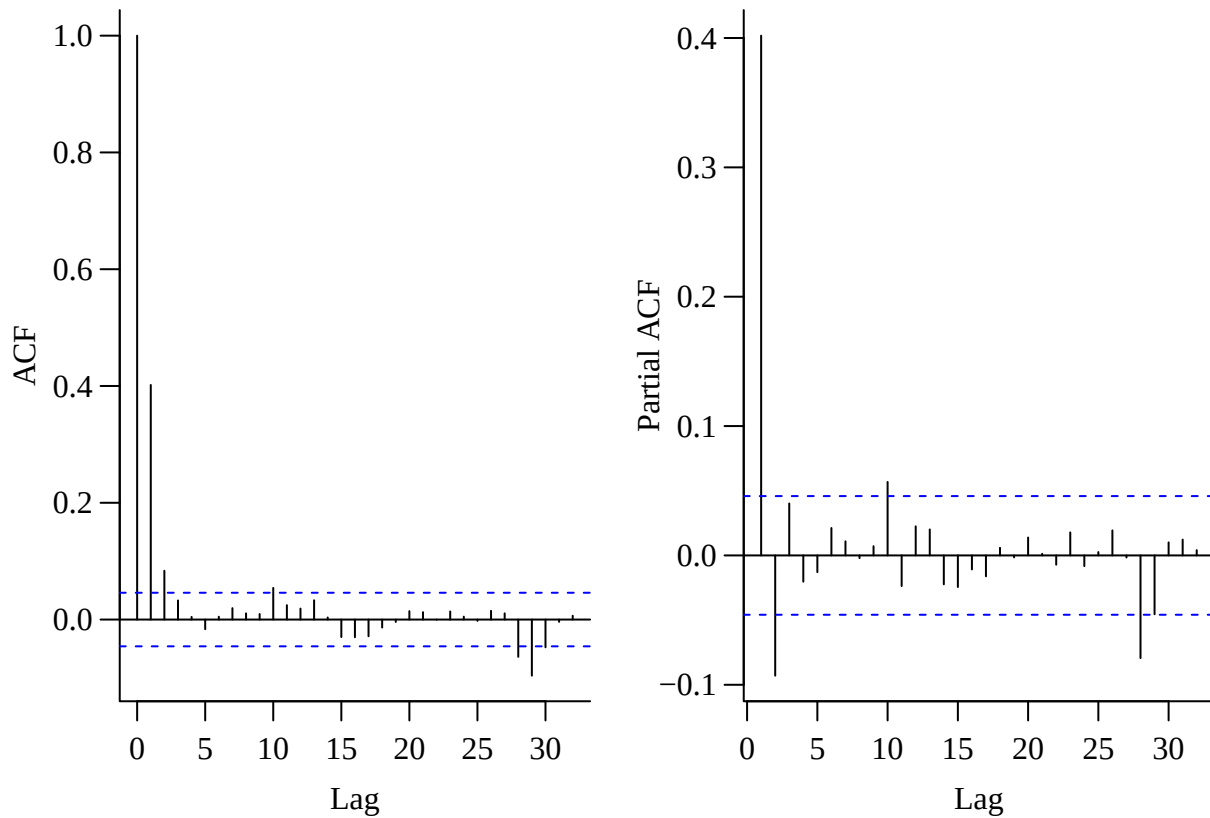
$$Y_t = \sqrt{W_t}.$$

```
y <- sqrt_wind
```

Identify the remaining temporal dependence

The harmonic regression accounts for the deterministic annual cycle but does not model serial dependence. As an exploratory step, we examine the ACF and PACF of the deseasonalized residuals. These plots help identify plausible low-order ARMA models for the stochastic component.

```
par(mfrow = c(1, 2), bty = "L", mar = c(3.5, 3.5, 1, 0.5),
    mgp = c(2, 0.7, 0), las = 1, family = "serif")
acf(res_sqrt, main = "ACF of deseasonalized residuals")
pacf(res_sqrt, main = "PACF of deseasonalized residuals")
```



```
par(mfrow = c(1, 1))
```

Jointly estimate seasonality and temporal dependence

The preliminary residual analysis is useful for model identification, but we do not need to estimate the final model in two stages. Instead, we fit

$$Y_t = s(t; \beta) + \eta_t,$$

where $s(t; \beta)$ contains the Fourier terms and η_t follows an ARMA process. Thus, the seasonal regression coefficients and ARMA parameters are estimated jointly by maximum likelihood. Based on the ACF/PACF, we compare several low-order candidate models.

```
# Candidate ARMA orders for the stochastic component
orders <- list(
  "AR(1)"      = c(1, 0, 0),
  "AR(2)"      = c(2, 0, 0),
  "ARMA(1,1)"  = c(1, 0, 1),
  "ARMA(2,1)"  = c(2, 0, 1)
)

# Jointly estimate harmonic seasonality and ARMA dependence
fits <- lapply(orders, function(order) {
  arima(y, order = order, xreg = X, include.mean = TRUE, method = "ML")
})
```

The two-step analysis above is used for model identification and diagnostics; the final model is estimated jointly.

Compare the candidate models

We compare the jointly fitted models using AIC and AICc. The parameter count for AICc includes all estimated parameters in the likelihood, including the regression coefficients, ARMA parameters, and innovation variance.

```
# Small-sample correction to AIC
AICc <- function(fit, n) {
  k <- attr(logLik(fit), "df")
  AIC(fit) + 2 * k * (k + 1) / (n - k - 1)
}

comparison <- data.frame(
  Model = names(fits),
  AIC = vapply(fits, AIC, numeric(1)),
  AICc = vapply(fits, AICc, numeric(1), n = length(y))
)

# Smaller AIC/AICc indicates better relative fit
comparison <- comparison[order(comparison$AICc), ]
comparison
```

```
##           Model      AIC      AICc
## ARMA(1,1) ARMA(1,1) 3577.322 3577.494
## ARMA(2,1) ARMA(2,1) 3578.903 3579.104
## AR(2)      AR(2)    3580.057 3580.229
## AR(1)      AR(1)    3593.756 3593.901
```

```
# Retain the model with the smallest AICc
best_name <- comparison$Model[1]
best_fit <- fits[[best_name]]
```

Diagnose the selected model

Model selection should be followed by residual diagnostics. After accounting for both seasonality and ARMA dependence, the remaining innovations should show little serial correlation and should be reasonably compatible with the assumed Gaussian innovation distribution. We examine the innovation ACF and Q-Q plot and use the Ljung-Box test to assess remaining serial dependence.

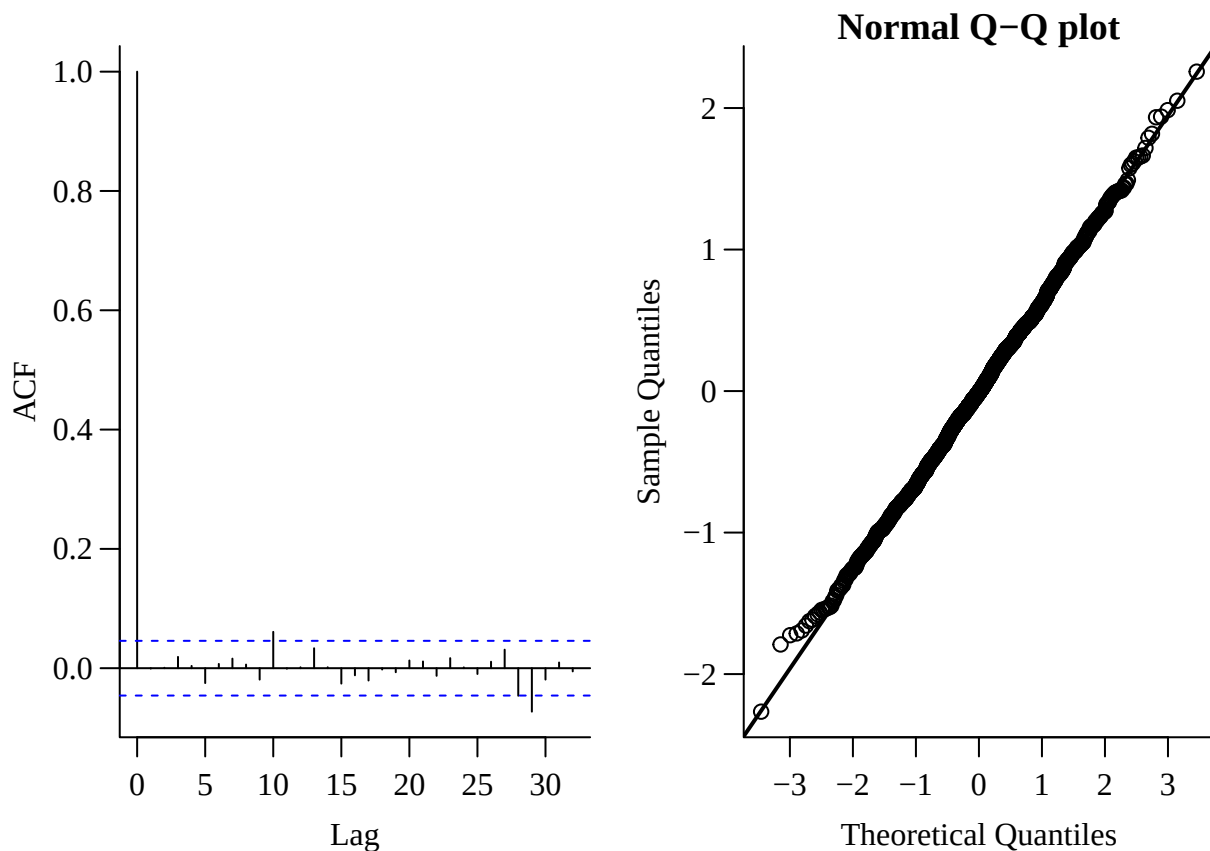
```
innovation <- residuals(best_fit)

# AR and MA orders of the selected model
p <- orders[[best_name]][1]
q <- orders[[best_name]][3]

par(mfrow = c(1, 2), bty = "L", mar = c(3.5, 3.5, 1, 0.5),
    mgp = c(2, 0.7, 0), las = 1, family = "serif")

acf(innovation, main = "ACF of innovations")

qqnorm(innovation, main = "Normal Q-Q plot")
qqline(innovation, lwd = 2)
```



```
par(mfrow = c(1, 1))

# Test for remaining serial correlation
Box.test(innovation, lag = 32, type = "Ljung-Box", fitdf = p + q)

##
## Box-Ljung test
##
## data: innovation
## X-squared = 32.695, df = 30, p-value = 0.3359
```

Forecast future wind speed

For forecasting, the future Fourier regressors must be evaluated at the future dates. The ARMA forecast then automatically combines the projected seasonal mean with the forecast of the stochastic component. Because the model is fitted to $Y_t = \sqrt{W_t}$, forecasts are initially obtained on the square-root scale.

```
# Forecast horizon
h <- 10

# Construct Fourier regressors for the next h days
future_date <- max(date) + seq_len(h)
future_time <- as.numeric(future_date - origin)
X_future <- harmonic_matrix(future_time, K)
```

```

# Forecast on the square-root scale
fc <- predict(best_fit, n.ahead = h, newxreg = X_future)

mean_sqrt <- as.numeric(fc$pred)
se_sqrt <- as.numeric(fc$se)

# Approximate 95% prediction limits on transformed scale
lower_sqrt <- mean_sqrt - qnorm(0.975) * se_sqrt
upper_sqrt <- mean_sqrt + qnorm(0.975) * se_sqrt

```

Simply squaring the transformed-scale forecast mean does not give the conditional mean of wind speed. Under the Gaussian transformed-scale predictive model,

$$Y_{n+h} \mid \mathcal{F}_n \sim N(m_h, v_h),$$

and $W_{n+h} = Y_{n+h}^2$, so

$$E(W_{n+h} \mid \mathcal{F}_n) = m_h^2 + v_h.$$

Thus, the variance term provides the appropriate mean correction when returning to the original scale.

```

# Conditional mean on the original wind-speed scale
mean_wind <- mean_sqrt^2 + se_sqrt^2

forecast_table <- data.frame(
  Date = future_date,
  Mean_sqrt = mean_sqrt,
  SE_sqrt = se_sqrt,
  Mean_wind = mean_wind)

forecast_table

```

##	Date	Mean_sqrt	SE_sqrt	Mean_wind
## 1	1970-01-01	4.137888	0.6401753	17.53194
## 2	1970-01-02	3.603557	0.7008263	13.47678
## 3	1970-01-03	3.504737	0.7029377	12.77731
## 4	1970-01-04	3.489164	0.7030145	12.66849
## 5	1970-01-05	3.489579	0.7030173	12.67140
## 6	1970-01-06	3.493132	0.7030174	12.69621
## 7	1970-01-07	3.497358	0.7030174	12.72575
## 8	1970-01-08	3.501773	0.7030174	12.75665
## 9	1970-01-09	3.506272	0.7030174	12.78818
## 10	1970-01-10	3.510822	0.7030174	12.82010

Because the forecast interval was constructed on the square-root scale, we need to transform it back to the original wind-speed scale. This requires some care because squaring is not monotone over the entire real line. If the interval on the square-root scale contains zero, the minimum possible value after squaring is zero. Otherwise, the lower bound is the smaller of the two squared endpoints. The upper bound is always the larger of the two squared endpoints.

```

# Back-transform the prediction interval from the square-root scale
# to the original wind-speed scale.
# If the interval contains 0, the lower bound on the original scale is 0.
lower_wind <- ifelse(lower_sqrt <= 0 & upper_sqrt >= 0, 0,

```

```

pmin(lower_sqrt^2, upper_sqrt^2))

# The upper bound is the larger of the two squared endpoints.
upper_wind <- pmax(lower_sqrt^2, upper_sqrt^2)

```

Plot the 10-Day Wind-Speed Forecast

The following plot compares the 10-day wind-speed forecasts with the most recent three months of observed wind speeds. The solid forecast line shows the point forecasts, while the shaded region represents the 95% prediction intervals. The vertical dashed line marks the end of the observed period and the beginning of the forecast period.

```

# -----
# Plot observations, forecasts, and 95% prediction intervals
# -----

# Show the last three months of observations
n_show <- 90
idx <- (length(wind_speed) - n_show + 1):length(wind_speed)

ylim <- range(c(wind_speed[idx], lower_wind, upper_wind))

par(bty = "L", mar = c(3.5, 3.5, 1, 0.5),
    mgp = c(2, 0.7, 0), las = 1, family = "serif")
plot(date[idx], wind_speed[idx], type = "l",
     xlim = c(min(date[idx]), max(future_date)), ylim = ylim,
     xlab = "Date", ylab = "Wind speed", main = "10-Day Wind-Speed Forecast")

# 95% prediction interval
polygon(c(future_date, rev(future_date)), c(lower_wind, rev(upper_wind)),
       border = NA, col = adjustcolor("grey70", alpha.f = 0.5))

# Observed series and point forecasts
lines(date[idx], wind_speed[idx], lwd = 1)
lines(future_date, mean_wind, lwd = 2)

# Mark the beginning of the forecast period
abline(v = max(date), lty = 3)

legend("topleft", legend = c("Observed", "Forecast", "95% prediction interval"),
     lty = c(1, 1, NA), lwd = c(1, 2, NA), pch = c(NA, NA, 15),
     col = c("black", "black", "grey70"), bty = "n")

```

